

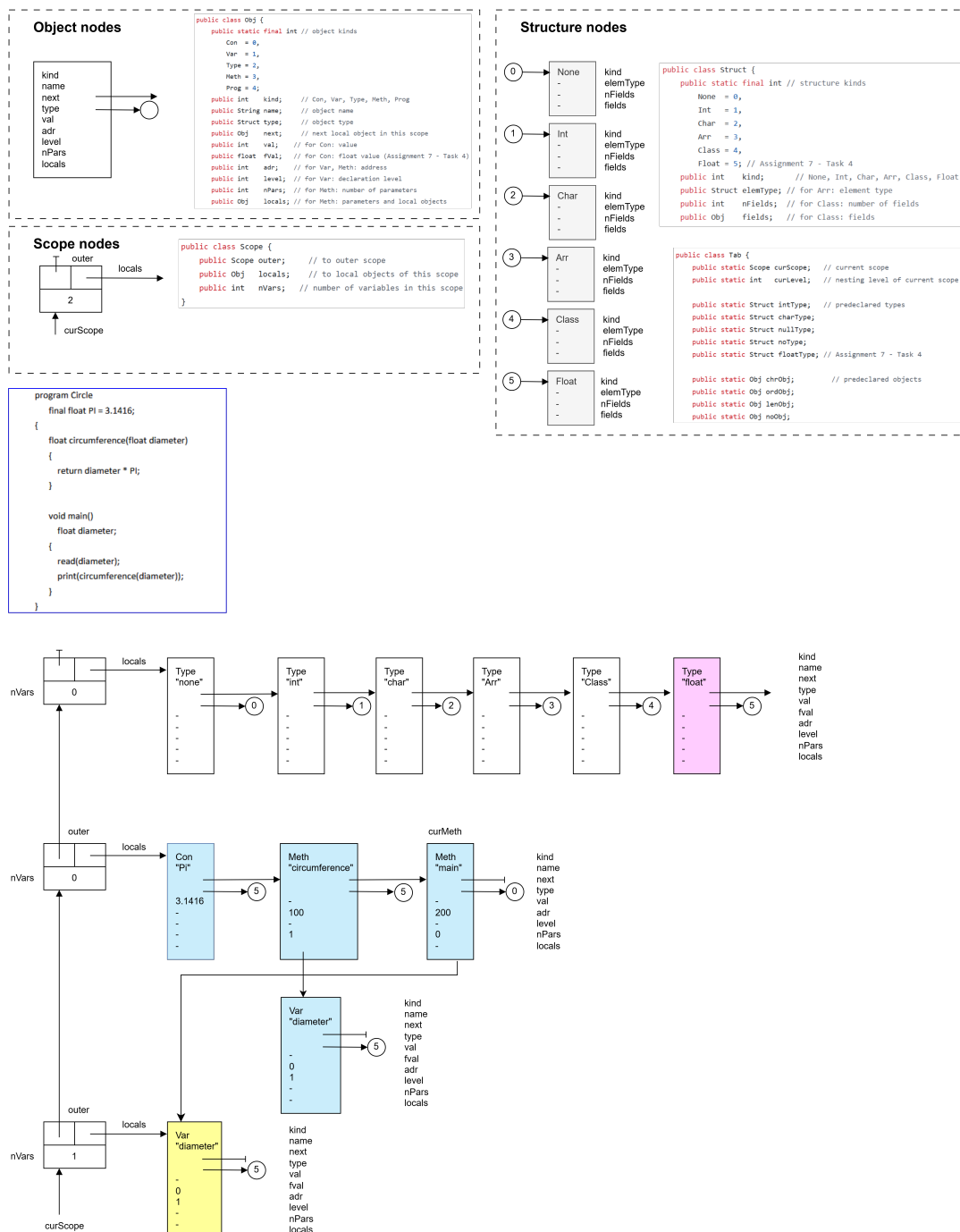
Assignment 7

Task 1 Symbol table extension

Describe necessary changes to the symbol table implementation in package MJ.SymTab to support this MicroJava extension. Show the structure of the symbol table after parsing the opening brace of method `main()`. Implementation is not necessary for this task – but see Task 4.

1. Symbol table with Float

After parsing the opening brace of `main()`, the symbol table contains:



2. Changes to MJ Code (Struct.java and Tab.java)

Struct.java

In struct.java we have to extend the struct class with a new kind Float = 5.

```
1  /* MicroJava Type Structures (HM 23-03-08)
2      =====
3      A type structure stores the propertires of a declared object's type.
4      -----*/
5
6  package MJ.SymTab;
7
8  public class Struct {
9      public static final int // structure kinds
10         None = 0,
11         Int = 1,
12         Char = 2,
13         Arr = 3,
14         Class = 4,
15         Float = 5;          // Assignment 7 - Task 4
16     public int kind;        // None, Int, Char, Arr, Class, Float
17     public Struct elemType; // for Arr: element type
18     public int nFields;     // for Class: number of fields
19     public Obj fields;      // for Class: fields
```

Tab.java

In Tab.java we have to:

- Add a corresponding structure instance (e.g., floatType) in Tab.java
- Insert float as a predeclared type into the symbol table
- Update the parser to recognize float as a valid type during type parsing
- Update the dumpStruct method

```
1  /* MicroJava Symbol Table (HM 23-10-02)
2      =====
3      This class maintains the symbol tyble of the MicroJava compiler.
4      It manages scopes and inserts and retrieves objects with their types.
5      -----*/
6
7  package MJ.SymTab;
8
9  import java.lang.*;
10 import MJ.*;
11
12 public class Tab {
13     public static Scope curScope;    // current scope
14     public static int curLevel;      // nesting level of current scope
15
16     public static Struct intType;    // predeclared types
17     public static Struct charType;
18     public static Struct nullType;
19     public static Struct noType;
20     public static Struct floatType; // Assignment 7 - Task 4
21
22     public static Obj chrObj;        // predeclared objects
23     public static Obj ordObj;
24     public static Obj lenObj;
25     public static Obj noObj;
```

```

1  //----- methods for dumping the symbol table -----
2
3  // Dumps a Struct
4  public static void dumpStruct(Struct type) {
5      String kind;
6      switch (type.kind) {
7          case Struct.Int: kind = "Int "; break;
8          case Struct.Char: kind = "Char "; break;
9          case Struct.Arr: kind = "Arr "; break;
10         case Struct.Class: kind = "Class"; break;
11         case Struct.Float: kind = "Float"; break; // Assignment 7 - Task 4
12         default: kind = "None";
13     }
14     //...
15 }

1  private static String getTypeName(Struct type) {
2      switch (type.kind) {
3          case Struct.Int: return "Int ";
4          case Struct.Char: return "Char ";
5          case Struct.Arr: return "Arr ";
6          case Struct.Class: return "Class";
7          case Struct.Float: return "Float"; // Assignment 7 - Task 4
8          default: return "None ";
9      }
10 }

1  //----- initialization of the symbol table -----
2
3  // Builds the universe
4  static {
5      curScope = new Scope();
6      curScope.outer = null;
7      curLevel = -1;
8
9      // create predeclared types
10     intType = new Struct(Struct.Int);
11     charType = new Struct(Struct.Char);
12     nullType = new Struct(Struct.Class);
13     noType = new Struct(Struct.None);
14     floatType = new Struct(Struct.Float); // Assignment 7 - Task 4
15     noObj = new Obj(Obj.Var, "???", noType);
16
17     // create predeclared objects
18     insert(Obj.Type, "int", intType);
19     insert(Obj.Type, "char", charType);
20     insert(Obj.Type, "float", floatType); // Assignment 7 - Task 4
21     insert(Obj.Con, "null", nullType);

```

Obj.java

```
1  /* MicroJava Symbol Table Objects (HM 23-10-02)
2      =====
3      Every named object in a program is stored in an Obj node.
4      Every scope has a list of objects declared in this scope.
5      -----*/
6
7  package MJ.SymTab;
8
9  public class Obj {
10      public static final int // object kinds
11          Con = 0,
12          Var = 1,
13          Type = 2,
14          Meth = 3,
15          Prog = 4;
16      public int kind; // Con, Var, Type, Meth, Prog
17      public String name; // object name
18      public Struct type; // object type
19      public Obj next; // next local object in this scope
20      public int val; // for Con: value
21      public float fVal; // for Con: float value (Assignment 7 - Task 4)
22      public int adr; // for Var, Meth: address
23      public int level; // for Var: declaration level
24      public int nPars; // for Meth: number of parameters
25      public Obj locals; // for Meth: parameters and local objects
26
27      public Obj(int kind, String name, Struct type) {
28          this.kind = kind; this.name = name; this.type = type;
29      }
30  }
```

Task 2 Type Checking (No mixed expressions)

Given the MicroJava language extension of Task 1, design changes in `MJ.SymTab` and `MJ.Parser` methods that check compatibility of data types in expressions, assignments, and method calls, allowing integer or floating-point operands, but not mixed expressions. Implementation is not required for this task.

1. Theoretical Background

MicroJava uses **name equivalence** to compare types: two types are equal if they point to the same `Struct` object (e.g., `x.type == y.type`). This is sufficient for primitive types and supports simple and efficient type checking.

With the addition of `float`, we must update the compiler to:

- Allow operations with operands of type `int + int` or `float + float`
- Reject all mixed expressions, e.g., `float + int` or `int = float`

Reference: 05-SymbolTable.pdf, slides 30–36 and 5.05-Typprüfungen.pdf

2. Task Solution

Changes are required in both `MJ.Parser` and `MJ.SymTab`:

MJ.Parser

Includes methods like `Expr()`, `Term()`, and `Statement()`. Type checking for assignments and expressions happens here. To reject mixed expressions, logic must be added in these methods.

MJ.SymTab

Refers to the symbol table classes `Tab.java`, `Struct.java`, etc. Adding support for `float` requires introducing a new `Struct.Float` constant and registering the type `"float"` in the universe scope via `Tab.floatType`.

Changes in `MJ.SymTab`

```
1 // Struct.java
2 public static final int Float = 5;
3
4 // Tab.java
5 public static Struct floatType = new Struct(Struct.Float);
6 insert(Obj.Type, "float", floatType);
```

Changes in `MJ.Parser`

```
1 // In Expr() or Term()
2 if (! (x.type.equals(Tab.intType) && y.type.equals(Tab.intType)) &&
3     ! (x.type.equals(Tab.floatType) && y.type.equals(Tab.floatType))) {
4     error("operands must be of same type (int or float)");
5 }
6
7 // In Statement()
8 if (!y.type.equals(x.type)) {
9     error("incompatible types in assignment");
10 }
```

These checks enforce strict separation of integer and floating-point types, as required in this task.

Planned changes in MJ.SymTab

To support the float data type, we need to introduce it into the compiler's symbol table and type system.

Struct.java

- We add a new type constant to represent floating-point numbers: `public static final int Float = 5;`.
- No changes are required for the `assignableTo()` or `compatibleWith()` methods. Since they both rely on the `equals()` method (which checks if the structure kind matches), they will naturally allow float to float assignment and comparisons, and automatically reject mixed types (like int to float).

Tab.java

- We will add a new predefined type variable: `public static Struct floatType;`.
- In the initialization block (`static ...`), we will initialize this new type: `floatType = new Struct(Struct.Float);`.
- We will insert the float type into the universe scope so it can be recognized as a valid type identifier in programs: `insert(Obj.Type, "float", floatType);`.
- We will update `dumpStruct()` and `getTypeName()` to handle `Struct.Float` for correct symbol table dumping and debugging output.

Planned changes in MJ.Parser

The syntax analyzer needs to be updated to type-check arithmetic and assignments correctly with the new type. By implementing these specific modifications, the compiler will be able to parse and type-check floating-point variables and expressions precisely according to the requirements, ensuring no implicit mixing of types occurs.

Expressions and Terms (`Expr()` and `Term()`)

- Currently, arithmetic operations enforce integer operands: `if (x.type != Tab.intType —— y.type != Tab.intType) error(...)`.
- We have to change this logic to verify that both operands are of the same numeric type (either both int or both float). The check will look something like this: `if (x.type != y.type —— (x.type != Tab.intType && x.type != Tab.floatType)) error("operands must be of the same numeric type");`.

Unary Minus (`Expr()`)

- The unary minus currently requires an integer. I will update it to allow both integer and float: `if (x.type != Tab.intType && x.type != Tab.floatType) error("numeric operand required");`.

Method Parameters (`ActPars()`)

- The logic in `ActPars()` checks if `(!ap.type.assignableTo(fp.type))`. Since `assignableTo()` correctly requires identical types for primitive values, this will automatically work for float parameters. No changes are needed here.

Assignments (`Statement()`)

- Assignments check if `(y.type.assignableTo(x.type))`. Similar to method parameters, this will automatically succeed for `float = float` and fail for `float = int` or `int = float`. Thus, no changes are required to enforce the "no mixed expressions" rule in assignments.

Built-in Functions (`read` and `print`)

- In `Statement()`, the `read` and `print` routines currently restrict arguments to `Tab.intType` or `Tab.charType`. I will expand these checks to also accept `Tab.floatType`.

private static Operand Expr()

```
1 // Expr = ['-'] Term {('+' | '-') Term}.
2 private static Operand Expr() {
3     Operand x;
4     if (sym == minus) {
5         scan();
6         x = Term();
7         if (x.type != Tab.intType && x.type != Tab.floatType) error("numeric operand required");
8         if (x.kind == Operand.Con) { // Assignment 7 - Task 4
9             x.val = - x.val;
10            x.fVal = - x.fVal;
11        } else {
12            Code.load(x);
13            Code.put(Code.neg);
14        }
15    } else {
16        x = Term();
17    }
18    while (sym == plus || sym == minus) {
19        int op = sym == plus ? Code.add : Code.sub;
20        scan();
21        Code.load(x);
22        Operand y = Term();
23        Code.load(y);
24        if (x.type != y.type || (x.type != Tab.intType && x.type != Tab.floatType)) // Assignment 7 - Task 4
25            error("operands must be of the same numeric type");
26        Code.put(op);
27    }
28    return x;
29 }
```

Strukturäquivalenz vs. Namensäquivalenz

Bei **Namensäquivalenz** werden zwei Typen als gleich betrachtet, wenn sie durch denselben Typnamen beschrieben werden. Im Compiler erfordert das lediglich eine Prüfung, ob sie durch denselben Strukturknoten repräsentiert werden. Die Typprüfung läuft also auf einen Vergleich hinaus, ob zwei Typzeiger auf denselben Knoten zeigen (`a.type == b.type`), was einfach und effizient ist.

Bei **Strukturäquivalenz** werden zwei Typen als gleich betrachtet, wenn sie die gleiche Struktur haben.

Bei Arrays erfordert das eine Prüfung ob sie denselben Elementtyp haben. Bei Klassen muss geprüft werden, ob sie die gleiche Anzahl von Feldern haben und ob die Typen der entsprechenden Felder strukturäquivalent sind, was auf einen rekursiven Vergleich hinausläuft. Obwohl Strukturäquivalenz mächtiger ist als Namensäquivalenz, ist sie aufwändiger und erfordert einen komplizierteren und auch ineffizienteren Vergleich strukturierter Typen. Auf Grund der Einfachheit (und auch besseren Verständlichkeit) verwenden die meisten modernen Programmiersprachen daher Namensäquivalenz. In Java (und auch in MicroJava) wird allerdings bei Arrays Strukturäquivalenz verwendet, weil Arraytypen in Java nicht durch einen Typnamen (z.B. `PersonArray`) beschrieben werden können, sondern nur durch eine Struktur (z.B. `Person[]`).

Task 3 Type Conversion (Int to Float)

Similar to Task 2, describe necessary changes to MJ.Parser and MJ.SymTab that allow automatic conversion from integer to floating-point operands in expressions, making the following example assignments valid.

1. Theoretical Background

Unlike Task 2, Task 3 allows for automatic (implicit) conversion from `int` to `float`. This follows the principle of *Widening*, where no information is lost. However, a conversion from `float` to `int` (*Narrowing*) remains forbidden to prevent precision loss.

In most programming languages, including Java and C, an `int` can be automatically promoted to a `float` when needed. This is known as a *widening conversion*. However, converting from `float` to `int` (a *narrowing conversion*) must be avoided unless explicitly cast, as it may lead to loss of precision.

Byte $\rightarrow$ *short* $\rightarrow$ *char* $\rightarrow$ *int* $\rightarrow$ *long* $\rightarrow$ *float* $\rightarrow$ *double*

In the MicroJava compiler, such conversions must be handled by extending the type checking logic so that mixed expressions like `float + int` are allowed, and the result type is `float`.

References: Slides 30–36 in 05-SymbolTable.pdf

Changes in MJ.SymTab

The central logic for type compatibility must be adjusted in the `Struct` class.

Adjustment of Compatibility Checks

The method `assignableTo(Struct dest)` in `Struct.java` should be modified to allow the widening transition:

```
1 public boolean assignableTo(Struct dest) {
2     // Identical types are always compatible
3     if (this.equals(dest)) return true;
4
5     // NEW for Task 3: int can be assigned to float
6     if (this.kind == Struct.Int && dest.kind == Struct.Float) {
7         return true;
8     }
9
10    return false; // float to int remains false
11 }
```

Changes in MJ.Parser

The parser must now correctly determine the resulting type during expression evaluation.

Arithmetic Expressions (Expr, Term)

When operands are mixed, the resulting type is promoted to `float`:

- `int op int` $\rightarrow$ `int`
- `float op float` $\rightarrow$ `float`
- `float op int` OR `int op float` $\rightarrow$ `float`

Analysis of the Example Program

- `q = m - K;`
`m - K` results in an `int`. Since `q` is a `float`, the assignment is **OK** due to implicit widening.
- `p = (R + 1) * 4;`
`R` is `float`, `1` is `int`. The expression evaluates to `float`. Assignment to `p` (`float`) is **OK**.
- `print(product(K));`
The method expects `float` and receives `int`. This is now **OK** via conversion.
- `print(address(p));`
The method expects `int` but receives `float`. This still results in an **Error** because narrowing is not permitted.

Summary of Implementation Steps

1. Extend `assignableTo` in `Struct.java` to handle `int → float`.
2. Update result type determination in `Parser.Expr()` and `Parser.Term()` (if any operand is `float`, the result is `float`).
3. Update `read()` and `print()` checks in the parser to permit `float` types.